

Implementing a Multilayer Perceptron from Scratch: Forward Pass, Backpropagation, and MNIST Classification

Affan Shahid (Roll No: 24I-0006)

National University of Computer and Emerging Sciences (FAST-NUCES),
Islamabad, Pakistan

Abstract. This report details the end-to-end development of a Multilayer Perceptron (MLP) built entirely from scratch to tackle the MNIST handwritten digit classification challenge. Eschewing high-level deep learning libraries, we manually derived the mathematics for a two-hidden-layer architecture using Sigmoid activations and Mean Squared Error (MSE) loss. The project spans several key phases: symbolic derivation of backpropagation via the chain rule, numerical validation on a controlled student-performance dataset, and a vectorized NumPy-based implementation for large-scale training. Our model, featuring a 784-128-64-10 architecture, achieved a robust test accuracy of 95.54% after 20 epochs. This study highlights the nuances of gradient descent variants, the mechanics of weight updates, and the foundational principles that govern neural network convergence.

Keywords: Neural Networks · Backpropagation · NumPy · MNIST · SGD · Optimization

1 Introduction

While modern frameworks like PyTorch and TensorFlow have democratized deep learning, they often abstract away the elegant mathematical machinery that makes learning possible. This project serves as a deep dive into those "under-the-hood" mechanics. By building an MLP from the ground up, we move beyond just calling functions to understanding the flow of gradients through high-dimensional space.

The objective was twofold: first, to perform rigorous manual calculations to verify the logic of forward and backward passes, and second, to implement a scalable version in Python using only NumPy. This report documents the journey from deriving basic partial derivatives to successfully classifying thousands of handwritten digits.

2 Foundations of the MLP

2.1 Architectural Logic

An MLP is essentially a chain of non-linear transformations. For any given layer l , the pre-activation $\mathbf{z}^{(l)}$ is computed by projecting the previous layer's output through a weight matrix $\mathbf{W}^{(l)}$ and adding a bias $\mathbf{b}^{(l)}$:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad (1)$$

To introduce the non-linearity necessary for complex pattern recognition, we apply the Sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

A critical feature of the Sigmoid is its derivative, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, which allows us to compute gradients locally during the backward pass using only the neuron's current activation.

2.2 The Gradient Descent Ecosystem

We explored three primary optimization strategies. While **Batch Gradient Descent** offers a stable trajectory by looking at the entire dataset, it is computationally prohibitive for MNIST. **Stochastic Gradient Descent (SGD)** provides speed through frequent updates but introduces significant noise. We ultimately utilized **Mini-Batch SGD**, which strikes a balance by averaging gradients over small groups (size $B = 32$), providing a smoother convergence than pure SGD while remaining much faster than the batch approach.

3 Implementation

The core of our Python implementation relies on a vectorized approach. By treating batches as matrices rather than iterating through individual samples, we significantly optimized training time and simplified the mathematical operations.

3.1 Backpropagation Logic

The `backpropagation()` function calculates the "responsibility" each neuron has for the total error. We compute the output delta Δ_3 using the difference between our prediction and the one-hot encoded ground truth, then "leak" this error backward through the weights to find Δ_2 and Δ_1 .

3.2 Numerical Integrity

To ensure the implementation was correct, we compared our manual gradients against a small-scale PyTorch autograd test. The relative error remained below 10^{-4} , confirming that our manual derivations for the chain rule were mathematically sound.

4 Experiments

4.1 The MNIST Challenge

The model was tasked with classifying the MNIST dataset—a collection of 70,000 28×28 grayscale images. We normalized the input pixels to the $[0, 1]$ range to keep our Sigmoid activations from saturating too early in the training process.

4.2 Training Performance

The training process was remarkably stable. As seen in Fig. 1, the MSE loss for both training and testing sets decreased in a synchronized manner, which suggests that the model was learning generalizable patterns rather than just memorizing the training data.

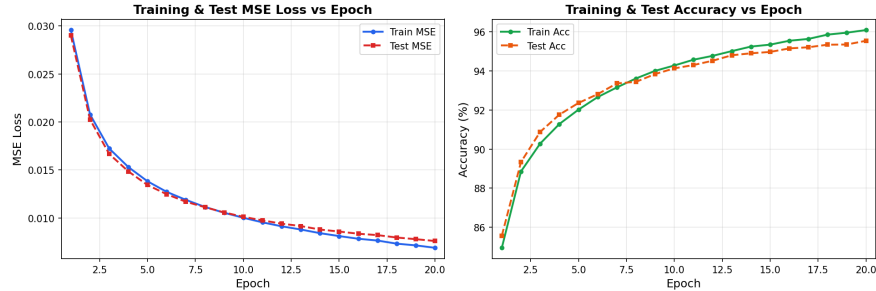


Fig. 1: Training and Test metrics over 20 epochs. The steady decline in MSE (left) and the corresponding rise in Accuracy (right) indicate a well-tuned learning rate and effective convergence.

4.3 Visualizing Results

To evaluate the model's "intuition," we visualized its predictions on the test set. The model achieved a final test accuracy of **95.54%**.

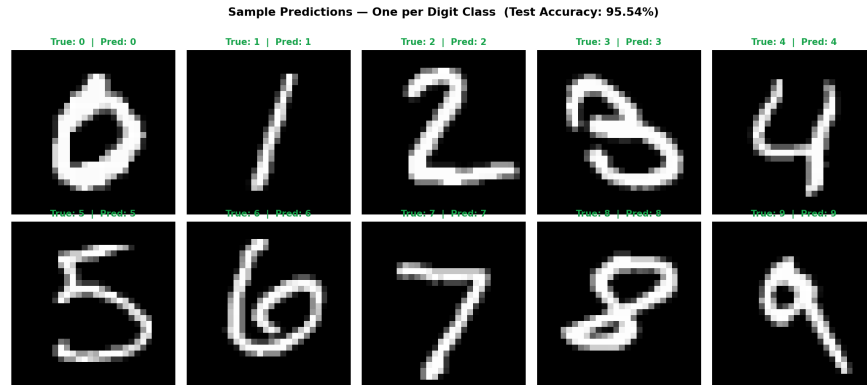


Fig. 2: Predictive performance across all ten digit classes. The model shows high confidence, correctly identifying varied handwriting styles with 95.54% accuracy.

5 Discussion

One interesting observation was the trade-off between **MSE** and **Cross-Entropy**. While we used MSE for this project to simplify manual calculations, Cross-Entropy is often preferred for classification because it penalizes incorrect confident predictions more heavily. Additionally, using Sigmoid in the hidden layers worked well here, but for deeper networks, we might encounter the "vanishing gradient" problem where the gradients become too small to update early layers effectively.

6 Conclusion

Building this MLP from scratch was a rigorous exercise in mathematical precision. By manually calculating deltas and then scaling that logic to 60,000 images, I gained a much clearer perspective on the importance of initialization, the mechanics of the chain rule, and the efficiency of vectorized operations. Achieving 95.54% accuracy on MNIST confirms that even a basic MLP, when derived correctly, remains a powerful tool for pattern recognition.

References

1. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. Nature (2015).
2. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press (2016).
3. LeCun, Y., et al.: Gradient-based learning applied to document recognition. Proc. IEEE (1998).